Глава 12. Многопоточность. RTOS

Ежелев Г. И.

2 мая 2026 г.

Управление задачами

• Весь код, который вы писали до этого - выполнялся последовательно. То есть был какой-то основной цикл в котором выполнялся весь код. • Такой подход называется Super loop

- Такой код быстрее писать и удобнее понимать
- Завис один узел - гарантированно зависла вся программа
- Сложно писать распределение нагрузки. Если регулятор нужно обновлять раз в 100 мс, раз в 10 мс нужно опрашивать датчик и раз в 30 мс отправлять пакеты экрану - на каждое действие должен быть свой таймер (а если частота важна - то прерывание).
- Многозадачность реализуется конечным автоматом.

Конечный автомат

Конечный автомат

Конечный автомат (Finite State Machine, FSM) - это программа, имеющая конечное число состояний и правил перехода между ними. В целом довольно простая мысль. Суть в том, как эти состояния реализовать.

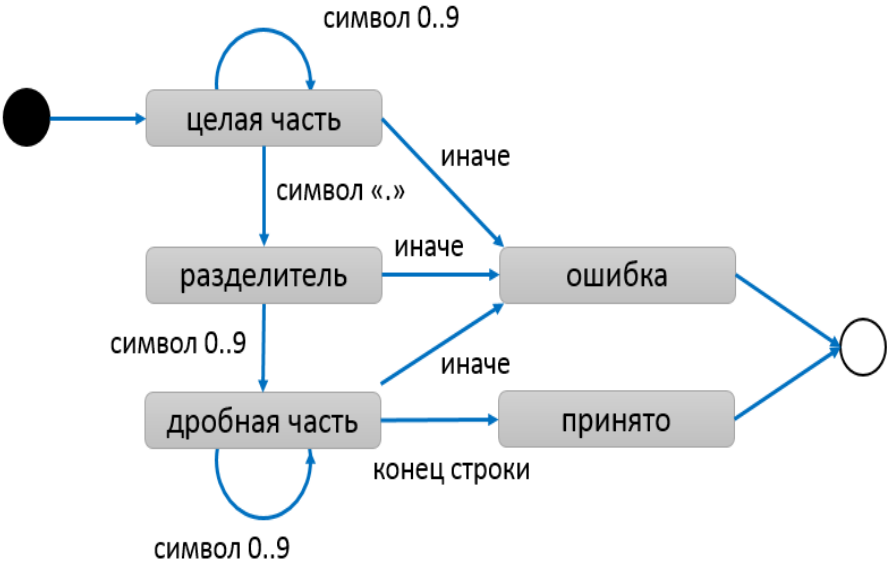
В случае робототехники:

- Цикл делится на 3 части: Listen-Think-Act (Слушай-Думай-Делай).
- Listen - первая часть (в самом начале), занимается обработкой датчиков (никаких чтений датчиков в других частях цикла - это костыль)
- Think - Конечный автомат. Он обычно делится на две части.
  • Первая - только выбор состояния на основании данных с датчиков. То есть набор условий:
  if(state == 1) [условия перехода из 1 -> {2,..., n}]
  ....
  if(state == n)[условия перехода из 1 -> {1,..., n-1}]

• Вторая часть - только выполнение состояний (без переходов между ними):
if(state == 1) [Действие в состоянии 1]
....
if(state == n)[Действие в состоянии n] • Такой подход позволяет не запутаться во всех состояниях, когда код разрастеться.

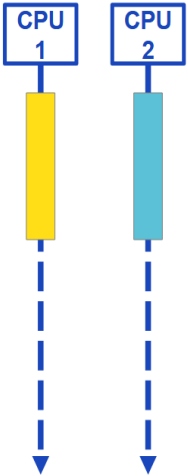
- Act - В конце - отдача приказов узлам робота (моторы, экранчик и т.п.)никаких моторов в середине кода!.
- Listen-Think-Act - это пример паттерна проектирования - это некоторый шаблон или идея, как надо реализовывать какой-то кусок кода (т.к. не вы первые догадались датчики читать и потом по данным с них двигаться).
- Многие паттерны звучат очевидно и скучно, но при этом почему-то люди всё равно их игнорируют и пишут плохой код ☺

Пример. Парсинг строки



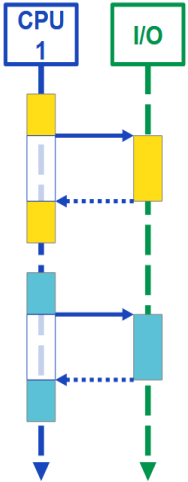
Управление задачами

- Два процессора
- Две задачи
- Параллельное исполнение
- Более сложное управление



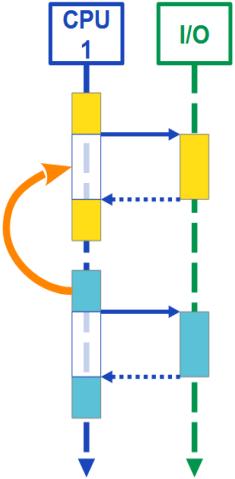
Управление задачами

- Один процессор
- Две задачи
- Ввод-вывод
- Процессор простаивает



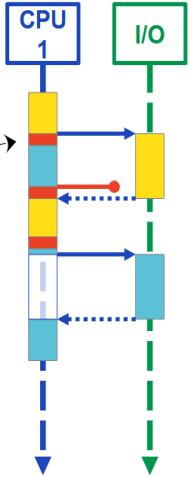
Управление задачами

- Один процессор
- Две задачи
- Ввод-вывод
- Процессор простаивает
  и это можно использовать



Управление задачами

- Один процессор
- Две задачи
- Ввод-вывод
- Переключение контекста
- Конкурентное исполнение
- Более сложное управление



Конкурентность

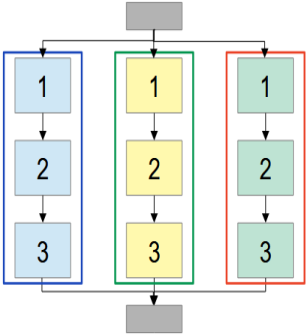
- Конкурентность - исполнение нескольких задач одновременно. Это не всегда значит, что задачи физических работают одновременно. Главное - они реализованы, как отдельные независимые задачи, которые не знают о существовании других.
- В случае, если одна задача отдает на время управление, программа переключается на исполнение другой задачи. В этот момент происходит переключение контекста - сохранение данных одной задачи и подгрузка данных следующей.
- Даже с одним ядром - такой подход имеет свои плюсы - вам не нужно думать, как распределять ресурсы между задачами (моторы раз в 100 мс, регуляторы раз в 10...). Каждая идейная задача - отдельная параллельная задача в коде, система сама переключит контекст когда надо.

Итог

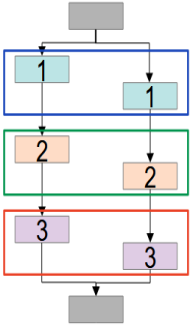
- Один процессор
- Много задач
- Многозадачность
- Конкурентное исполнение

Варианты архитектур

- распараллеливание
  - обработчик выполняет все этапы одной задачи



- конвейерная обработка
  - обработчик выполняет один этап всех задач



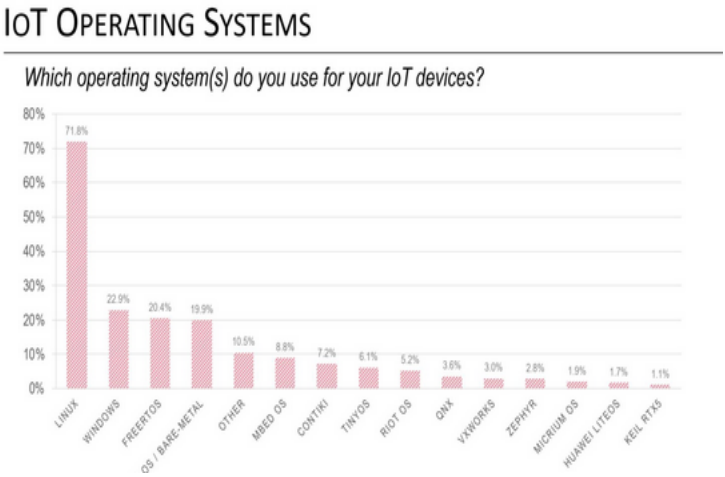
RTOS

Все манипуляции с процессами (например переключение контекста) осуществляет планировщик(Scheduler) - часть операционной системы.
•Операционные системы реального времени (Real Time Operating System, RTOS) - это ОС, которые ставят в приоритет не равномерность распределения ресурсов между задачами и удобство пользователя, а временную детерминированность, т.е. реакция на событие произойдет в строго(почти) определенное время.
Пример такой системы - FREERTOS, состоящей из:

- Планировщика(scheduler) - решает, какая задача выполняется сейчас
- Диспетчера задач (Task manager) - следит за состоянием задачи, памятью и тп.
- Управление памятью (Heap manager) - управление данными задач (их целых 4-5 штук)
- Механизмы синхронизации - очереди, семафоры, уведомления, группы событий.

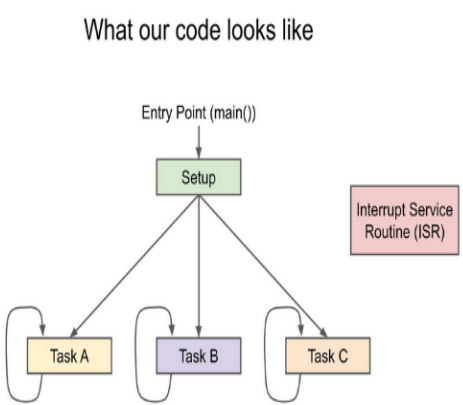
Популярность RTOS

По опросу Eclipse Foundation для IoT (Internet of Things) устройств после Linux и Windows- самой популярной основой стал FREERTOS, обогнав разработку без операционной системы (Bare Metal)

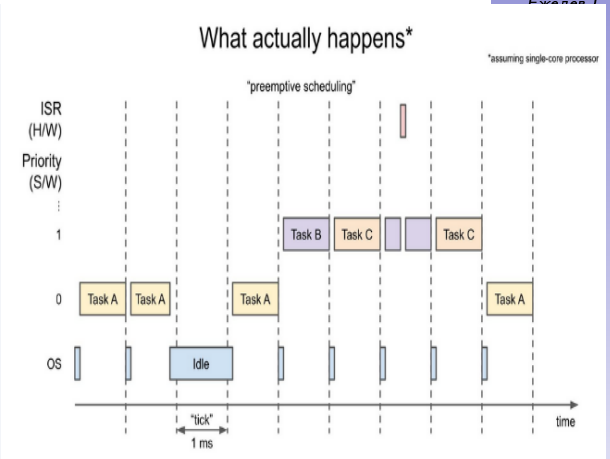


Задачи во FREERTOS

Задача - это какая-то функция, обычно реализованная как (почти)бесконечный цикл, планировщик их сам нарезает:



Как мы видим задачи в коде



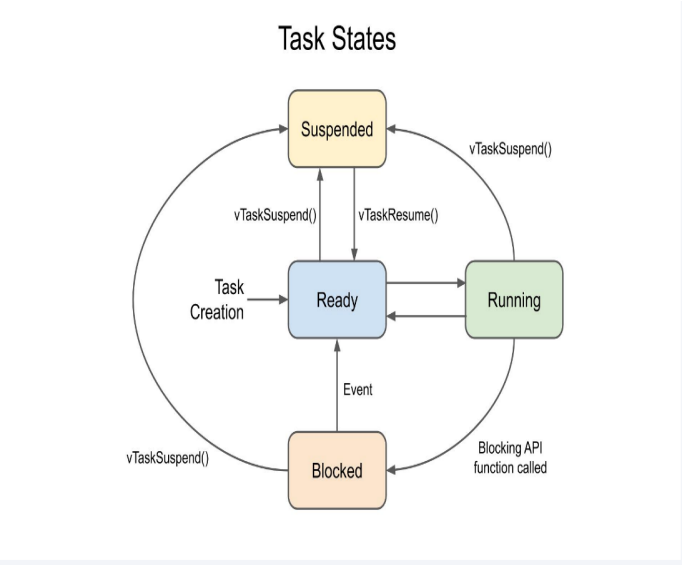
Их реальное исполнение

Переключение задач

- Каждую миллисекунду FreeRtos пробуждает ядро по тому самому прерыванию SysTick. Каждый период между пробуждениями ОС так и называется - "тик".
- Задачи переключаются операционкой в следующих ситуациях:
- По приоритету задач. В новый тик есть заждавшаяся задача того же приоритета или любая задача более высокого приоритета. Это реализовано через массив очередей(у каждого уровня приоритетов своя очередь) - pxReadyTasksLists[configMAX_PRIORITIES].
- Вытеснение (Preemption). Если неважная задача А ставиться на паузу прерыванием, например приемом пакета UART (а прерывания никто не отменял), а после приема пакета в очередь попадает более важная задача Б, то RTOS сразу последует в Б, не завершая А.

- Ожидание. Мы можем поставить задачу в ожидание, пока какая-то другая задача что-то не сделает, что пробудит нашу задачу.
- Блокировка. Задачи могут обмениваться данными. Если задача попробует запросить данные у другой, которых еще нет, она заблокируется, пока данные не появятся, а в это время будут исполняться другие таски

Состояния задач

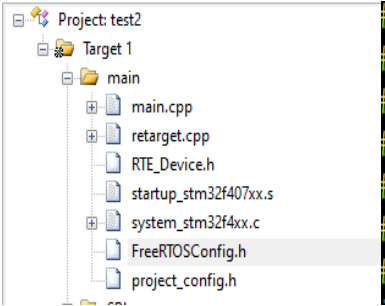


Состояния задач

- Ready - После создания задача попадает в это состояние. Такие задачи готовы к исполнению и ждут внимания ядра
- Running - выполнение
- Blocked - заблокирована. Например вызвана задержка, или задача попыталась запросить данные от другой задачи, а их нет (Блокировка) или задача ожидает событие (Ожидание). В этом состоянии задача лежит в планировщике.
- Suspended - приостановлена. Более радикальная мера, задача выбрасывается из планировщика. Нужна для отключения какого-то куска логики. Зачем нужно? => чтобы сохранить контекст и данные самой задачи.

сборка проекта FREERTOS

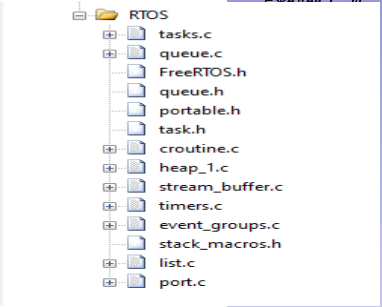
Все файлы операционки уже есть в проекте. проверьте что все нужные файлы добавлены в keil:



FreeRtosConfig.h

```
#pragma once
#include "misc.h"
#include "project_config.h"
#include "FreeRTOS.h"
#include "task.h"
#include "queue.h"
#include "portable.h"
```

Не забудьте подключить



Все файлы операционки

Закоментировать SysTick в нашем Time.h, FREERTOS его тоже использует для синхронизации

Советую посмотреть файл FreeRtosConfig.h (он выложен в репе, скопируйте к себе, в проекте он возможно неправильный)

Создание задачи

```
BaseType_t xTaskCreate(
    TaskFunction_t pvTaskCode,
    const char * const pcName,
    configSTACK_DEPTH_TYPE usStackDepth,
    void *pvParameters,
    UBaseType_t uxPriority,
    TaskHandle_t *pxCreatedTask
);
```

```
void f1(void *pvParams)
{
  while(true)
  {
    //your code...
  }
}
```

main.cpp

- pcName - имя задачи (просто строка)
- usStackDepth - глубина стека задачи (об этом позже)
- pvParameters - объект для передачи параметров если ничего не передаем - NULL
- uxPriority - приоритет задачи для планировщика - больше ⇔ важнее
- pxCreatedTask - структура для управления задачей извне, можно передать NULL
- Отметим, что в FREERTOS есть одна всегда запущенная задача с низшим приоритетом - Idle Task (простой) (она даже есть выше на диаграмме what actually happens).
- Нельзя использовать RETURN!
- А как переключать задачи?